

Analyzing data on the level of cognitive processes - ESCoP 2025

Gidon T. Frischkorn, Ruhi Bhanap, and Isabel Courage

Solutions: Exercise 2: Hierarchical Models & Signal Detection Theory

```
# empty environment
rm(list = ls())

# load libraries
library(brms)
library(tidyverse)
library(here)
```

A - Fit Bayesian GLMs to the aggregated data

1. Load the **aggregated data** of experiment 2 of Williams et al. (2022) `exercise2_Williams2022_exp2_aggregated.txt`.

This data set contains aggregated performance indices from a Change Detection Task with Set Size 8 and a between subject manipulation `isAdaptive`. The manipulation entailed a prompt pushing participants to respond equally often with `same` and `different` to achieve a less biased response pattern.

```
data_Williams2022_agg <- read.table(here("data", "exercise2_Williams2022_exp2_aggregated.txt"))
head(data_Williams2022_agg)
```

##		id	isAdaptive	prop_correct	K	dprime	crit
## 1	0rjs8zk3f8a1c51		no	0.6666667	2.666667	0.9707840	0.49096234
## 2	1ntjctjk7c30h8e		yes	0.6400000	2.240000	0.7201744	0.09521913
## 3	1o34yg68q5f0082		no	0.7000000	3.200000	1.5997169	0.95082764
## 4	1p2orrruwowkyfk		yes	0.8600000	5.760000	2.2012469	0.19369466
## 5	1sj18cefeen6xdq		no	0.6266667	2.026667	0.7562075	0.56297078
## 6	1t9ccets6d4jmh1		no	0.7111111	3.377778	1.1321919	0.18377351

2. Fit Bayesian GLMs with either proportion correct, Cowan's K or dprime as the dependent variable.

Use the `brm` function to specify that the dependent variable should vary as a function of the `isAdaptive` between-subject manipulation. Set appropriate priors for the `b` coefficients for the respective models. Use the `default_prior` function to get information which priors are set as a default by `brms`. Set the `sample_prior` option to `TRUE` when fitting the model. We need this option in order to perform hypothesis tests with the `hypothesis()` function later on.

```
# check default priors
default_prior(prop_correct ~ isAdaptive, data = data_Williams2022_agg)
```

##		prior	class	coef	group	resp	dpar	nlpar	lb	ub
##		(flat)	b							
##		(flat)	b	isAdaptive	yes					
##	student_t(3, 0.7, 2.5)		Intercept							
##	student_t(3, 0, 2.5)		sigma						0	

```

##          source
##          default
## (vectorized)
##          default
##          default

# change priors for parameters of class b
my_priors_pc <- prior(normal(0,0.25), class = b)

# fit model
fit_pc <- brm(
  formula = bf(prop_correct ~ 1 + isAdaptive, sigma ~ 1 + isAdaptive),
  data = data_Williams2022_agg,
  family = gaussian(),
  cores = 4,
  prior = my_priors_pc,
  sample_prior = TRUE,
  backend = "cmdstanr",
  refresh = 0,
  file = here("models", "model_sdt_glm_pc"),
  file_refit = "never"
)

# check default priors
default_prior(K ~ isAdaptive, data = data_Williams2022_agg)

##          prior      class      coef group resp dpar nlpar lb ub
##          (flat)         b
##          (flat)         b isAdaptiveyes
## student_t(3, 2.8, 2.5) Intercept
## student_t(3, 0, 2.5)      sigma
##          source
##          default
## (vectorized)
##          default
##          default

# change priors for parameters of class b
my_priors_CowansK <- prior(normal(0,1), class = b)

# fit model
fit_CowansK <- brm(
  formula = bf(K ~ 1 + isAdaptive, sigma ~ 1 + isAdaptive),
  data = data_Williams2022_agg,
  family = gaussian(),
  cores = 4,
  prior = my_priors_CowansK,
  sample_prior = TRUE,
  backend = "cmdstanr",
  refresh = 0,
  file = here("models", "model_sdt_glm_k"),
  file_refit = "never"
)

# check default priors
default_prior(dprime ~ isAdaptive, data = data_Williams2022_agg)

```

```
##               prior      class      coef group resp dpar nlpar lb ub
##               (flat)         b
##               (flat)         b isAdaptiveyes
## student_t(3, 1.1, 2.5) Intercept
## student_t(3, 0, 2.5)      sigma
## source
## default
## (vectorized)
## default
## default

# change priors for parameters of class b
my_priors_dprime <- prior(normal(0,1), class = b)

# fit model
fit_dprime <- brm(
  formula = bf(dprime ~ 1 + isAdaptive, sigma ~ 1 + isAdaptive),
  data = data_Williams2022_agg,
  family = gaussian(),
  cores = 4,
  prior = my_priors_dprime,
  sample_prior = TRUE,
  backend = "cmdstanr",
  refresh = 0,
  file = here("models", "model_sdt_glm_dprime"),
  file_refit = "never"
)
```

3. Quantify the strength of evidence for or against an effect of the manipulation.

The `hypothesis` function allows to specify to-be-tested hypotheses for any GLM fitted with `brms`. For this you need to pass the `brmfit` object and a vector of character strings specifying the hypothesis you want to test. Usually you formulate the hypothesis in form of a null hypothesis. In order to use this function you need to set in your model the `sample_prior` option to `TRUE`.

```
# proportion correct
hypothesis(fit_pc,
  "isAdaptiveyes = 0")

## Hypothesis Tests for class b:
##               Hypothesis Estimate Est.Error CI.Lower CI.Upper Evid.Ratio Post.Prob
## 1 (isAdaptiveyes) = 0      0.04      0.01      0.01      0.07      0.41      0.29
## Star
## 1      *
## ---
## 'CI': 90%-CI for one-sided and 95%-CI for two-sided hypotheses.
## '*': For one-sided hypotheses, the posterior probability exceeds 95%;
## for two-sided hypotheses, the value tested against lies outside the 95%-CI.
## Posterior probabilities of point hypotheses assume equal prior probabilities.

# Cowan's K
hypothesis(fit_CowansK,
  "isAdaptiveyes = 0")

## Hypothesis Tests for class b:
##               Hypothesis Estimate Est.Error CI.Lower CI.Upper Evid.Ratio Post.Prob
## 1 (isAdaptiveyes) = 0      0.62      0.23      0.16      1.07      0.1      0.09
```

```
## Star
## 1 *
## ---
## 'CI': 90%-CI for one-sided and 95%-CI for two-sided hypotheses.
## '*': For one-sided hypotheses, the posterior probability exceeds 95%;
## for two-sided hypotheses, the value tested against lies outside the 95%-CI.
## Posterior probabilities of point hypotheses assume equal prior probabilities.

# dPrime
hypothesis(fit_dprime,
            "isAdaptiveyes = 0")

## Hypothesis Tests for class b:
##           Hypothesis Estimate Est.Error CI.Lower CI.Upper Evid.Ratio Post.Prob
## 1 (isAdaptiveyes) = 0      0.08      0.1      -0.1      0.27      7.34      0.88
## Star
## 1
## ---
## 'CI': 90%-CI for one-sided and 95%-CI for two-sided hypotheses.
## '*': For one-sided hypotheses, the posterior probability exceeds 95%;
## for two-sided hypotheses, the value tested against lies outside the 95%-CI.
## Posterior probabilities of point hypotheses assume equal prior probabilities.
```

B - Fit Bayesian-hierarchical SDT model to the data

1. Load the **non-aggregated data** of experiment 2 of Williams et al. (2022): `exercise2_Williams2022_exp2.txt`.

This data contains more detailed data on the number of hits and false alarms for trials with targets and foils as test stimuli.

```
data_Williams2022 <- read.table(here("data", "exercise2_Williams2022_exp2.txt"))
head(data_Williams2022)
```

```
##           id isAdaptive was_same_trial sum_saysame sum_correct n_trials
## 1 0rjs8zk3f8a1c51      no              0          37         188        225
## 2 0rjs8zk3f8a1c51      no              1         112         112        225
## 3 1ntjctjk7c30h8e     yes              1         136         136        225
## 4 1ntjctjk7c30h8e     yes              0          73         152        225
## 5 1o34yg68q5f0082      no              0           9         216        225
## 6 1o34yg68q5f0082      no              1          99          99        225
## prop_correct
## 1      0.8355556
## 2      0.4977778
## 3      0.6044444
## 4      0.6755556
## 5      0.9600000
## 6      0.4400000
```

2. Specify the non-linear formula to fit an SDT model with this data.

Use the `brmsformula` function to specify the non-linear formula implementing an SDT model in a GLM. Predict both `dprime` and the criterion by the `isAdaptive` manipulation.

```
sdt_formula <- bf(
  sum_saysame | trials(n_trials) ~ -crit + was_same_trial * dprime,
  dprime ~ 1 + isAdaptive + (1 | id),
  crit ~ 1 + isAdaptive + (1 | id),
```

```
nl = TRUE
)
```

3. Fit the SDT model to the data.

Pass the `brmsformula` to the `brm` function and fit a binomial model with a **probit** link function.

Again consider specifying adequate priors for the parameters and plot the model predictions (hint: `conditional_effects()`). Use the `hypothesis` function to test on which parameters of the SDT model the `isAdaptive` manipulation has an effect.

Priors:

```
# check default priors
default_prior(sdt_formula, data = data_Williams2022, family = binomial(link = "probit"))
```

```
##           prior class           coef group resp dpar  nlpar lb ub
##           (flat)    b              Intercept              crit
##           (flat)    b isAdaptiveeyes              crit
##           (flat)    b isAdaptiveeyes              crit
## student_t(3, 0, 2.5) sd              id              crit 0
## student_t(3, 0, 2.5) sd              id              crit 0
## student_t(3, 0, 2.5) sd      Intercept      id              crit 0
##           (flat)    b              dprime
##           (flat)    b      Intercept              dprime
##           (flat)    b isAdaptiveeyes              dprime
## student_t(3, 0, 2.5) sd              dprime 0
## student_t(3, 0, 2.5) sd              id              dprime 0
## student_t(3, 0, 2.5) sd      Intercept      id              dprime 0
##           source
##           default
## (vectorized)
## (vectorized)
##           default
## (vectorized)
## (vectorized)
##           default
## (vectorized)
## (vectorized)
##           default
## (vectorized)
## (vectorized)
##           default
## (vectorized)
## (vectorized)
```

```
# adapt priors
my_priors_sdt <- prior(normal(0,1), class = b, nlpar = crit) +
  prior(normal(0,1), class = b, nlpar = dprime)
```

Model:

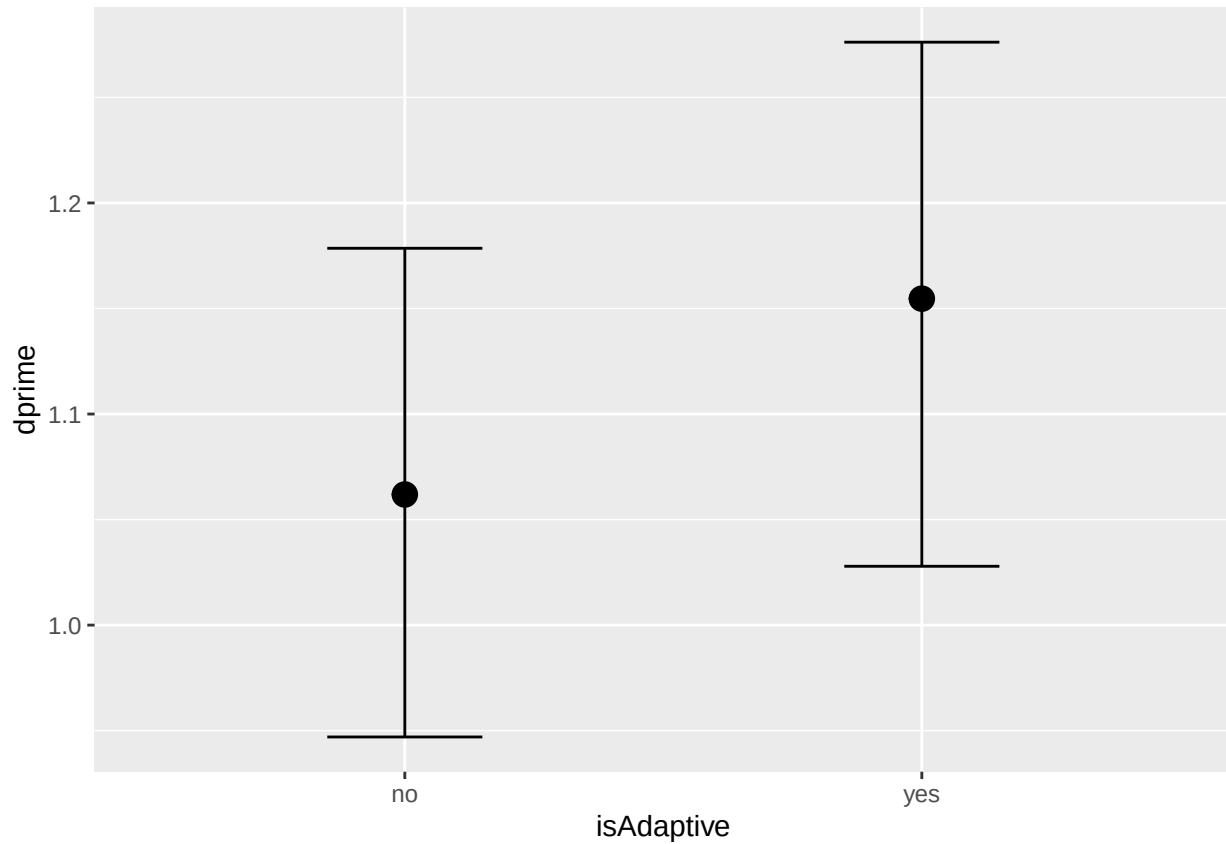
```
# fit model
fit_sdt <- brm(
  formula = sdt_formula,
  data = data_Williams2022,
  family = binomial(link = "probit"),
  prior = my_priors_sdt,
  sample_prior = TRUE,
  cores = 4,
```

```

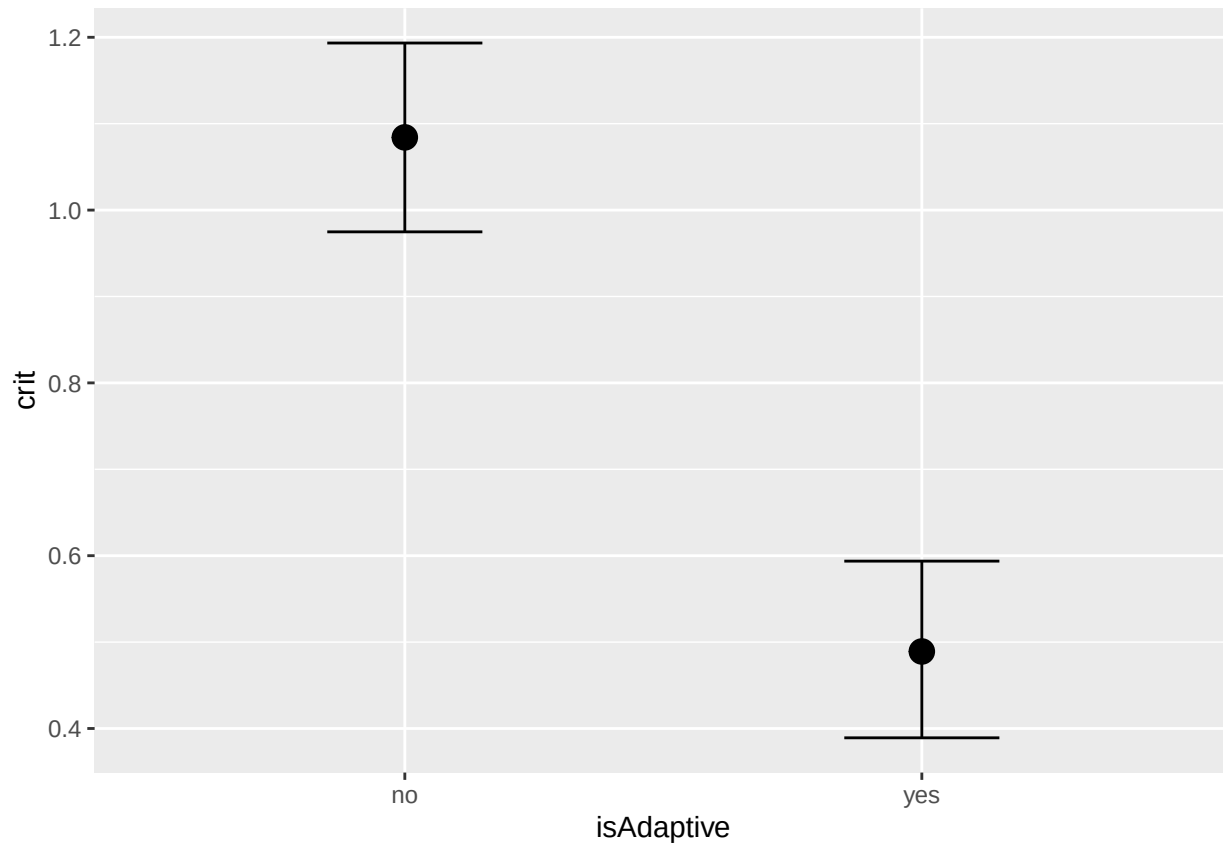
refresh = 0,
backend = "cmdstanr",
file = here("models", "model_sdt"),
file_refit = "never"
)

# plot model predicted effects
conditional_effects(fit_sdt, nlpar = "dprime", effects = "isAdaptive")

```



```
conditional_effects(fit_sdt, nlpar = "crit", effects = "isAdaptive")
```



Hypotheses:

test if manipulation had effects on SDT parameters

```
hypothesis(fit_sdt,
  c("dprime_isAdaptiveyes = 0",
    "crit_isAdaptiveyes = 0"))
```

Hypothesis Tests for class b:

##	Hypothesis	Estimate	Est.Error	CI.Lower	CI.Upper	Evid.Ratio
## 1	(dprime_isAdaptive... = 0	0.09	0.09	-0.08	0.26	6.45
## 2	(crit_isAdaptiveyes) = 0	-0.59	0.08	-0.74	-0.44	0.00

Post.Prob Star

##	Post.Prob	Star
## 1	0.87	
## 2	0.00	*

2 0.00 *

'CI': 90%-CI for one-sided and 95%-CI for two-sided hypotheses.

'*': For one-sided hypotheses, the posterior probability exceeds 95%;

for two-sided hypotheses, the value tested against lies outside the 95%-CI.

Posterior probabilities of point hypotheses assume equal prior probabilities.